

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 6**



Graph dan Tree

Oleh:

Nadya Nadhir Soraya

NIM. 2510817120022

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
JUNI 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 6

Laporan Praktikum Algoritma & Struktur Data Modul 6: Graph dan Tree ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Nadya Nadhir Soraya
NIM : 2510817120022

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani
NIM. 2310817310009

Ir. Muhamamd Alkaff, S,Kom, M.Kom,
Ph.D
NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN	ii
DAFTAR ISI.....	iii
DAFTAR GAMBAR	v
DAFTAR TABEL	vi
SOAL 1	1
Konversi Adjacency List ke Adjacency Matrix	1
A. Source Code	1
B. Output Program	3
C. Pembahasan	4
SOAL 2	6
Konversi Adjacency Matrix ke Adjacency List	6
A. Source Code	6
B. Output Program	8
C. Pembahasan	8
SOAL 3	10
BFS: Jarak Terpendek Keluar Labirin	10
A. Source Code	10
B. Output Program	13
C. Pembahasan	13
SOAL 4	15
DFS: Menghitung Semua Jalur Keluar Labirin	15
A. Source Code	15
B. Output Program	17
C. Pembahasan	17
SOAL 5	20
Tree Traversal	20
A. Source Code	20
B. Output Program	23
C. Pembahasan	23

TAUTAN GITHUB25

DAFTAR GAMBAR

Gambar 1. Screenshot Output Program Soal 1	3
Gambar 2. Screenshot Output Program Soal 2	8
Gambar 3. Screenshot Output Program Soal 3	13
Gambar 4. Screenshot Output Program Soal 4	17
Gambar 5. Screenshot Output Program Soal 5	23

DAFTAR TABEL

Tabel 1. Source Code Soal 1	1
Tabel 2. Source Code Soal 2	6
Tabel 3. Source Code Soal 3	10
Tabel 4. Source Code Soal 4	15
Tabel 5. Source Code Soal 5	20

SOAL 1

Konversi Adjacency List ke Adjacency Matrix

A. Source Code

Tabel 1. Source Code Soal 1

1	<code>#include <iostream></code>
2	
3	<code>using namespace std;</code>
4	
5	<code>int main() {</code>
6	<code> int n;</code>
7	<code> cin >> n;</code>
8	
9	<code> char vertex[10];</code>
10	<code> for (int i = 0; i < n; i++) {</code>
11	<code> cin >> vertex[i];</code>
12	<code> }</code>
13	
14	<code> int matrix[10][10];</code>
15	
16	<code> for (int i = 0; i < n; i++) {</code>
17	<code> for (int j = 0; j < n; j++) {</code>
18	<code> matrix[i][j] = 0;</code>
19	<code> }</code>
20	<code> }</code>
21	
22	<code> int m;</code>
23	<code> cin >> m;</code>
24	
25	<code> for (int i = 0; i < m; i++) {</code>

```

26     char u, v;
27     int w;
28     cin >> u >> v >> w;
29
30     int indeks_u = -1;
31     for (int j = 0; j < n; j++) {
32         if (vertex[j] == u) {
33             indeks_u = j;
34             break;
35         }
36     }
37
38     int indeks_v = -1;
39     for (int j = 0; j < n; j++) {
40         if (vertex[j] == v) {
41             indeks_v = j;
42             break;
43         }
44     }
45
46     if (indeks_u != -1 && indeks_v != -1) {
47         matrix[indeks_u][indeks_v] = w;
48     }
49 }
50
51 cout << "Adjacency Matrix:" << endl;
52
53 cout << " ";
54 for (int i = 0; i < n; i++) {
55     cout << " " << vertex[i];

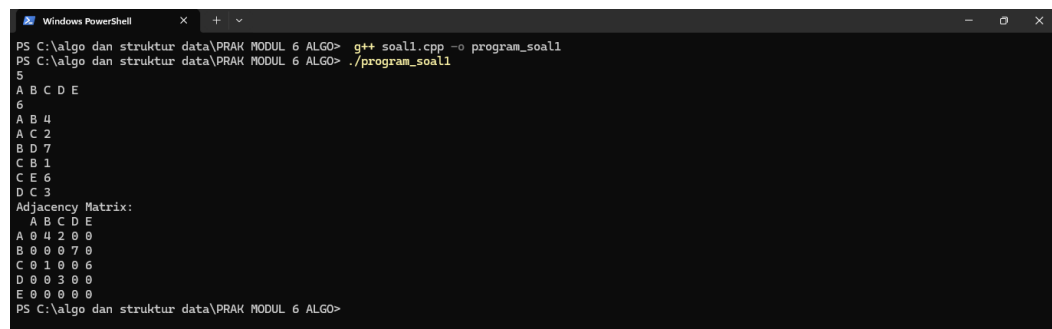
```

```

56     }
57     cout << endl;
58
59     for (int i = 0; i < n; i++) {
60         cout << vertex[i];
61
62         for (int j = 0; j < n; j++) {
63             cout << " " << matrix[i][j];
64         }
65         cout << endl;
66     }
67
68     return 0;
69 }

```

B. Output Program



```

Windows PowerShell
PS C:\algo dan struktur data\PRAK MODUL 6 ALGO> g++ soal1.cpp -o program_soal1
PS C:\algo dan struktur data\PRAK MODUL 6 ALGO> ./program_soal1
S
A B C D E
S
A B 4
A C 2
B D 7
C B 1
C E 6
D C 3
Adjacency Matrix:
A B C D E
A 0 4 2 0 0
B 0 0 0 7 0
C 0 1 0 0 6
D 0 0 3 0 0
E 0 0 0 0 0
PS C:\algo dan struktur data\PRAK MODUL 6 ALGO>

```

Gambar 1. Screenshot Output Program Soal 1

C. Pembahasan

Algoritma ini bertujuan untuk mengubah representasi graph dari bentuk daftar sisi (adjacency list) menjadi matriks 2 dimensi (adjacency matrix). **Mekanismenya** adalah menggunakan teknik linear search untuk mencari indeks dari label vertex. Jadi, karena input vertex di soal berupa karakter huruf sedangkan matriks 2 dimensi membutuhkan indeks berupa angka mutlak, algoritma ini akan mengecek elemen array vertex mulai dari indeks 0 lalu membandingkannya dengan target karakter asal dan karakter tujuan. Jika masih belum ketemu, program maju ke indeks 1, indeks 2, dan seterusnya secara berurutan. Proses ini hanya akan berhenti jika target huruf yang dicari sudah ditemukan, untuk kemudian indeks tersebut dipakai sebagai acuan posisi baris dan kolom.

Untuk **cara kerjanya**, pertama algoritma mengeksekusi blok kode inisialisasi dengan mendeklarasikan array 2D dan mengisi seluruh elemennya dengan angka 0 menggunakan perulangan bersarang. Angka 0 ini dipakai sebagai asumsi dasar bahwa belum ada sisi atau jalur yang menghubungkan antar vertex. Setelah inisialisasi selesai, algoritma membaca input edge satu per satu secara iteratif. Saat menemukan input sisi, linear search tadi dieksekusi. Ketika indeks array baris dan kolomnya sudah didapat, matriks pada koordinat tersebut akan ditimpa nilainya dengan bobot yang diberikan. **Asumsi yang dibuat** di sini adalah graph bersifat directed atau berarah. Sehingga, mekanisme penimpaan nilai bobot hanya dilakukan searah dari baris asal ke kolom tujuan, sedangkan arah sebaliknya dibiarkan tetap bernilai 0. Algoritma ini juga berasumsi tidak ada jalur yang kembali ke titiknya sendiri, sehingga diagonal utama matriks selalu bernilai 0.

Untuk **analisis kompleksitas waktu**, kita bisa melihat dari alur perulangannya. Proses inisialisasi awal dan pencetakan matriks sama-sama menggunakan perulangan bersarang yang memakan eksekusi sebanyak N dikali N . Selanjutnya, saat membaca input jalur (*edge*) sebanyak M kali, algoritma melakukan *linear search* untuk mencari indeks baris dan kolom yang masing-

masing butuh maksimal N iterasi. Sehingga, tahap pemetaan ini memakan operasi sebesar M dikali N . Kesimpulannya, kompleksitas waktunya adalah $O(N^2 + N \times M)$. Karena batasan N maksimal hanya 10, total iterasinya sangatlah sedikit. Program dijamin berjalan kilat dan aman dari *Time Limit* 1.0 s.

Selanjutnya untuk **analisis kompleksitas memori**, kode ini berjalan murni secara iteratif sehingga bebas dari tumpukan ruang rekursif. Alokasi memori hanya dipesan satu kali di awal untuk mendeklarasikan array 1 dimensi berukuran 10 dan array 2 dimensi statis 10x10. Karena kebutuhan ruang terbesarnya hanya sebatas ukuran matriks N dikali N , maka kompleksitas ruangnya adalah $O(N^2)$. Pemakaian memori matriks 10x10 ini sangatlah irit, sehingga program dijamin tidak akan mengalami *crash* dan pasti aman 100% dari batasan *Memory Limit* 64 MB.

Kalau dilihat dari tabel contoh kasus, terdapat 5 vertex yaitu A, B, C, D, dan E. Huruf-huruf ini diletakkan ke dalam array vertex secara berurutan mulai dari indeks 0. Ketika algoritma masuk ke iterasi untuk membaca edge "A B 4", algoritma mengecek elemen mulai dari indeks 0 dan langsung menemukan karakter A di indeks 0. Kemudian algoritma mengecek lagi untuk titik tujuan dan menemukan karakter B di indeks 1. Dengan indeks tersebut, blok kode langsung mengubah elemen pada matriks baris 0 dan kolom 1 menjadi bernilai 4. Lalu untuk input edge "D C 3", karakter D dicek satu per satu hingga ditemukan di indeks 3, dan karakter C ditemukan di indeks 2, sehingga elemen matriks di baris 3 dan kolom 2 ditimpa dengan nilai 3. Karena tidak ada edge berbalik arah dari B ke A, maka elemen baris 1 kolom 0 secara otomatis tetap bernilai 0 sesuai inisialisasi awal. Jika algoritma sudah mengecek semua edge hingga ujung data, blok kode untuk mencetak matriks bisa dieksekusi.

SOAL 2

Konversi Adjacency Matrix ke Adjacency List

A. Source Code

Tabel 2. Source Code Soal 2

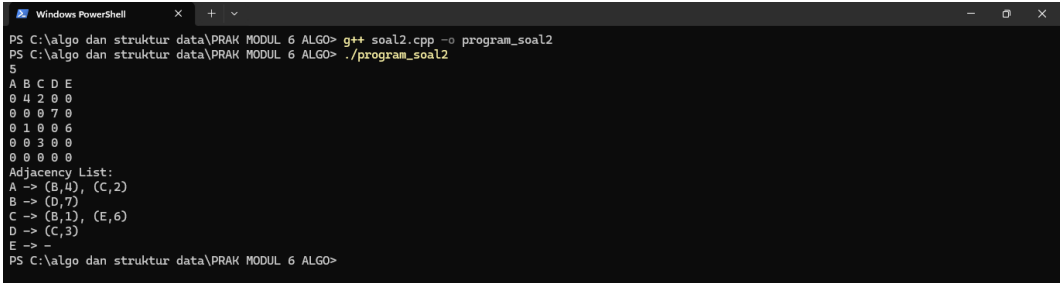
1	<code>#include <iostream></code>
2	
3	<code>using namespace std;</code>
4	
5	<code>int main() {</code>
6	<code> int n;</code>
7	<code> cin >> n;</code>
8	
9	<code> char vertex[10];</code>
10	<code> for (int i = 0; i < n; i++) {</code>
11	<code> cin >> vertex[i];</code>
12	<code> }</code>
13	
14	<code> int matrix[10][10];</code>
15	<code> for (int i = 0; i < n; i++) {</code>
16	<code> for (int j = 0; j < n; j++) {</code>
17	<code> cin >> matrix[i][j];</code>
18	<code> }</code>
19	<code> }</code>
20	
21	<code> cout << "Adjacency List:" << endl;</code>
22	
23	<code> for (int i = 0; i < n; i++) {</code>
24	<code> cout << vertex[i] << " ->";</code>
25	

```

26         bool pertama = true;
27
28         for (int j = 0; j < n; j++) {
29             if (matrix[i][j] > 0) {
30
31                 if (pertama == false) {
32                     cout << ",";
33                 }
34
35                 cout << " (" << vertex[j] << "," <<
matrix[i][j] << ")";
36                 pertama = false;
37             }
38         }
39
40         if (pertama == true) {
41             cout << " -";
42         }
43
44         cout << endl;
45     }
46
47     return 0;
48 }

```

B. Output Program



```
PS C:\algo dan struktur data\PRAK MODUL 6 ALGO> g++ soal2.cpp -o program_soal2
PS C:\algo dan struktur data\PRAK MODUL 6 ALGO> ./program_soal2
5
A B C D E
0 4 2 0 0
0 0 0 7 0
0 1 0 0 6
0 0 3 0 0
0 0 0 0 0
Adjacency List:
A -> (B,4), (C,2)
B -> (D,7)
C -> (B,1), (E,6)
D -> (C,3)
E -> -
PS C:\algo dan struktur data\PRAK MODUL 6 ALGO>
```

Gambar 2. Screenshot Output Program Soal 2

C. Pembahasan

Algoritma ini bertujuan untuk mengubah representasi graph dari bentuk matriks 2 dimensi (adjacency matrix) menjadi daftar sisi (adjacency list). **Mekanismenya adalah** melakukan pengecekan elemen secara berurutan di dalam matriks. Karena struktur matriksnya sudah diberikan di input, algoritma ini akan mengecek elemen matriks secara iteratif dari indeks kolom 0 hingga ujung batas N untuk setiap baris indeks vertex asal. Proses pengecekan satu per satu ini digunakan untuk mencari nilai elemen matriks yang lebih besar dari 0, yang menandakan bahwa ada jalur keluar dari titik tersebut beserta nilai bobotnya.

Untuk cara kerjanya, pertama algoritma mengeksekusi blok kode inisialisasi dengan mendeklarasikan array 1 dimensi untuk menampung label vertex dan array 2 dimensi untuk menampung isi matriks, lalu membaca melalui perulangan bersarang. Setelah semua data tersimpan, algoritma mengeksekusi perulangan bersarang lagi untuk mulai mencetak adjacency list. Untuk mengatur format pencetakan koma, algoritma menggunakan variabel boolean bernama pertama yang diatur menjadi true di awal pengecekan setiap baris. Saat mengecek kolom satu per satu, jika ditemukan angka matriks lebih dari 0, program langsung mencetak tujuan dan bobotnya, lalu mengubah status boolean menjadi false. Jika ada sisi lain yang ditemukan di baris yang sama, status false tersebut akan memicu algoritma untuk mencetak tanda koma sebagai pemisah. Namun, jika algoritma sudah mengecek elemen dari indeks awal hingga ujung dan status boolean tetap

true, itu menandakan vertex asal tidak memiliki edge keluar sama sekali, sehingga kode akan otomatis mencetak tanda strip (-).

Untuk **analisis kompleksitas waktu**, dilihat dari alur perulangannya. Pertama, algoritma melakukan perulangan bersarang sebesar N dikali N untuk membaca dan menyimpan input matriks. Kedua, terdapat perulangan bersarang lagi dengan ukuran yang sama untuk mengecek isi matriks sekaligus mencetak format *adjacency list*-nya. Karena total operasinya didominasi oleh perulangan tersebut tanpa ada proses pencarian tambahan, kesimpulan kompleksitas waktunya adalah $O(N^2)$. Berhubung nilai N maksimal di soal hanya 10, total eksekusinya sangat sedikit dan dijamin aman dari *Time Limit* 1.0 s.

Selanjutnya **untuk analisis memori**, program ini berjalan murni secara iteratif tanpa melibatkan tumpukan rekursif. Alokasi memori hanya dipesan sekali di awal untuk array 1 dimensi berukuran 10 dan array 2 dimensi statis berukuran 10×10 . Karena ruang memori yang terpakai hanya dikunci konstan mengikuti luas matriks maksimal sesuai batasan soal, maka kompleksitas ruangnya adalah $O(N^2)$. Ukuran array ini sangat kecil sehingga program terhindar dari *crash* otomatis dan pasti lolos batasan *Memory Limit* 64 MB.

Kalau dilihat dari soal, terdapat 5 vertex yaitu A, B, C, D, dan E yang berurutan ditampung dari indeks 0 sampai 4. Saat algoritma masuk ke iterasi baris indeks 0 untuk vertex A, algoritma mengecek elemen satu per satu dari kolom 0 hingga ujung. Algoritma kemudian mendeteksi angka lebih dari 0 pada kolom 1 yang bernilai 4, dan kolom 2 yang bernilai 2. Karena indeks kolom merepresentasikan titik tujuan, program menggunakan indeks tersebut untuk mencetak output (B,4) dan (C,2). Sebaliknya, saat algoritma mengeksekusi baris indeks 4 untuk vertex E, algoritma mengecek elemen kolom satu per satu dan hanya menemukan nilai 0 hingga akhir iterasi. Karena tidak ada edge keluar yang terdeteksi, nilai boolean tidak berubah sehingga blok kode otomatis merespons dengan mencetak tanda strip pada hasil E -> -. **Hasil keluaran ini terbukti sama dan dengan output pada modul.**

SOAL 3

BFS: Jarak Terpendek Keluar Labirin

A. Source Code

Tabel 3. Source Code Soal 3

```
1  #include <iostream>
2  #include <queue>
3
4  using namespace std;
5
6  struct Koordinat {
7      int baris;
8      int kolom;
9  };
10
11 int main() {
12     int r, c;
13     cin >> r >> c;
14
15     int grid[105][105];
16     int jarak[105][105];
17
18     for (int i = 0; i < r; i++) {
19         for (int j = 0; j < c; j++) {
20             cin >> grid[i][j];
21             jarak[i][j] = -1;
22         }
23     }
24
25     int start_r, start_c;
```

```

26     cin >> start_r >> start_c;
27
28     int finish_r, finish_c;
29     cin >> finish_r >> finish_c;
30
31     queue<Koordinat> antrean;
32
33     antrean.push({start_r, start_c});
34     jarak[start_r][start_c] = 0;
35
36     int arah_baris[] = {-1, 1, 0, 0};
37     int arah_kolom[] = {0, 0, -1, 1};
38
39     while (!antrean.empty()) {
40         Koordinat current = antrean.front();
41         antrean.pop();
42
43         if (current.baris == finish_r &&
44 current.kolom == finish_c) {
45             break;
46         }
47
48         for (int i = 0; i < 4; i++) {
49             int next_baris = current.baris +
50 arah_baris[i];
51             int next_kolom = current.kolom +
52 arah_kolom[i];
53
54             if (next_baris >= 0 && next_baris < r
55 &&

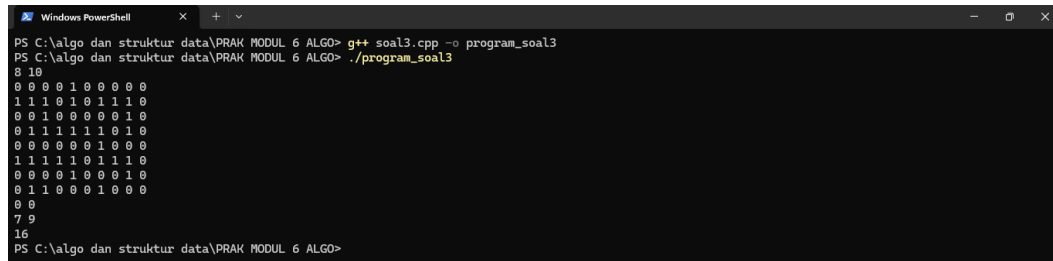
```

```

52         next_kolom >= 0 && next_kolom < c
53         &&
54         grid[next_baris][next_kolom] == 0
55         &&
56         jarak[next_baris][next_kolom] == -
57         1) {
58             jarak[next_baris][next_kolom] =
59             jarak[current.baris][current.kolom] + 1;
60             antrean.push({next_baris,
61             next_kolom});
62         }
63     }
64     cout << jarak[finish_r][finish_c] << "\n";
65     return 0;
66 }

```

B. Output Program



```
PS C:\algo dan struktur data\PRAK MODUL 6 ALGO> g++ soal3.cpp -o program_soal3
PS C:\algo dan struktur data\PRAK MODUL 6 ALGO> ./program_soal3
8
10
0 0 0 0 1 0 0 0 0 0
1 1 1 0 1 0 1 1 1 0
0 0 1 0 0 0 0 0 1 0
0 1 1 1 1 1 1 0 1 0
0 0 0 0 0 1 0 0 0
1 1 1 1 0 1 1 1 0
0 0 0 1 0 0 0 1 0
0 1 1 0 0 1 0 0 0
0 0
7
9
16
PS C:\algo dan struktur data\PRAK MODUL 6 ALGO>
```

Gambar 3. Screenshot Output Program Soal 3

C. Pembahasan

Algoritma ini bertujuan menentukan jumlah langkah minimum dari titik awal menuju tujuan menggunakan Breadth-First Search (BFS), yakni algoritma penelusuran graf secara melebar. Pada kasus ini, grid diibaratkan sebagai graf: sel jalan yang bernilai 0 disebut *vertex*, dan pergerakan 4 arah sekitarnya adalah *edge*. Menggunakan struktur antrean (*queue*) dengan prinsip First In First Out (FIFO), jadi algoritma ini tidak menelusuri satu jalur lurus sampai mentok ke jalan buntu. Sebaliknya, pencarian mengeksplorasi ruang kosong secara merata ke segala arah. Algoritma akan memproses semua kotak yang berjarak 1 langkah terlebih dahulu, baru kemudian bergeser ke kotak berjarak 2 langkah, 3 langkah, dan seterusnya secara berurutan. Karena sifatnya yang menyebar bertahap per tingkat (*level per level*), BFS dipastikan akurat untuk mencari *shortest path*. Saat antrean menyentuh titik tujuan untuk pertama kalinya, rute tersebut otomatis dijamin sebagai rute dengan jarak **tempuh paling pendek sehingga pencarian bisa langsung dihentikan.**

Untuk cara kerjanya, Secara teknis, algoritma menyiapkan array grid untuk peta dan array jarak yang isinya di-set -1 semua sebagai penanda kotak belum dikunjungi. Titik awal dimasukkan ke antrean dengan nilai jarak 0. Selama antrean belum kosong, program mengambil koordinat terdepan lalu mengecek 4 arah sekitarnya (atas, bawah, kiri, kanan). Kalau koordinat *next* itu masih di dalam

batas matriks, berupa jalan yang bisa dilewati (0), dan belum dikunjungi (-1), program akan mengupdate nilai jaraknya menjadi jarak kotak saat ini ditambah 1, lalu memasukkannya ke antrean baru. Asumsi yang dipakai di sini adalah pergerakan murni hanya horizontal dan vertikal tanpa ada langkah diagonal.

Untuk **analisis kompleksitas waktu**, algoritma memproses inisialisasi awal dan penelusuran BFS sebanding dengan luas petaknya. Dalam skenario terburuk (*worst case*), setiap sel jalan maksimal hanya masuk ke dalam antrean dan dicek 4 arahnya persis satu kali. Karena total operasinya murni bergantung pada jumlah sel, kompleksitas waktunya adalah $O(R \times C)$. Mengingat batasan R dan C maksimal hanya 100, eksekusi program paling mentok berada di kisaran 10.000 iterasi. Proses ini berjalan sangat cepat sehingga dipastikan aman dari *Time Limit* 1.0 s.

Selanjutnya untuk memori, algoritma ini berjalan murni secara iteratif tanpa menumpuk ruang rekursif sama sekali. Alokasi memori hanya dipakai untuk mendeklarasikan array statis berukuran 105x105 dan sebuah variabel antrean yang isinya dijamin tidak akan melebihi total sel. Karenanya, kesimpulan kompleksitas ruangnya adalah $O(R \times C)$. Penggunaan memori ini sangatlah irit, program terhindar dari otomatis *crash*, dan dipastikan lolos dari batasan *Memory Limit* 64 MB.

Jika diuji menggunakan contoh kasus 8x10, program berjalan dari koordinat awal (0,0) yang diberi nilai jarak 0. Saat mengecek 4 *next*, arah kiri dan atas otomatis ditolak karena keluar batas array, sedangkan arah bawah di koordinat (1,0) ditolak karena berupa tembok. Algoritma hanya berhasil merambat ke arah kanan di (0,1) lalu menimpa nilai jaraknya menjadi 1. Proses rambatan ini terus menyebar. Rute cabang yang masuk ke jalan buntu otomatis berhenti diproses, menyisakan rute utama yang menjalar hingga akhirnya mendeteksi target di (7,9). Berhubung petak sebelumnya sudah mencapai jarak 15, jarak di titik target ditimpa menjadi 16, dan program langsung menggunakan perintah *break* untuk keluar dari pencarian. Hasil akhirnya terbukti akurat dan sesuai dengan modul.

SOAL 4

DFS: Menghitung Semua Jalur Keluar Labirin

A. Source Code

Tabel 4. Source Code Soal 4

1	<code>#include <iostream></code>
2	<code>using namespace std;</code>
3	
4	<code>int r, c;</code>
5	<code>int grid[10][10];</code>
6	<code>bool visited[10][10];</code>
7	<code>int finish_r, finish_c;</code>
8	<code>int total_jalur = 0;</code>
9	
10	<code>int arah_baris[] = {-1, 1, 0, 0};</code>
11	<code>int arah_kolom[] = {0, 0, -1, 1};</code>
12	
13	<code>void dfs(int current_baris, int current_kolom) {</code>
14	<code> if (current_baris == finish_r && current_kolom</code>
	<code>== finish_c) {</code>
15	<code> total_jalur++;</code>
16	<code> return;</code>
17	<code> }</code>
18	
19	<code> visited[current_baris][current_kolom] = true;</code>
20	
21	<code> for (int i = 0; i < 4; i++) {</code>
22	<code> int next_baris = current_baris +</code>
	<code>arah_baris[i];</code>

```

23         int next_kolom = current_kolom +
    arah_kolom[i];
24
25         if (next_baris >= 0 && next_baris < r &&
26             next_kolom >= 0 && next_kolom < c &&
27             grid[next_baris][next_kolom] == 0 &&
28             visited[next_baris][next_kolom] ==
    false) {
29
30             dfs(next_baris, next_kolom);
31         }
32     }
33
34     visited[current_baris][current_kolom] = false;
35 }
36
37 int main() {
38     cin >> r >> c;
39
40     for (int i = 0; i < r; i++) {
41         for (int j = 0; j < c; j++) {
42             cin >> grid[i][j];
43             visited[i][j] = false;
44         }
45     }
46
47     int start_r, start_c;
48     cin >> start_r >> start_c;
49     cin >> finish_r >> finish_c;
50

```

51	dfs(start_r, start_c);
52	cout << total_jalur << "\n";
53	
54	return 0;
55	}

B. Output Program

```

PS C:\algo dan struktur data\PRAK MODUL 6 ALGO> g++ soal4.cpp -o program_soal4
PS C:\algo dan struktur data\PRAK MODUL 6 ALGO> ./program_soal4
5 6
0 0 0 1 0 0
0 1 0 0 0 1
0 1 0 1 0 0
0 0 0 1 1 0
1 1 0 0 0 0
0 0
4 5
4
PS C:\algo dan struktur data\PRAK MODUL 6 ALGO>

```

Gambar 4. Screenshot Output Program Soal 4

C. Pembahasan

Algoritma ini bertujuan untuk menghitung total semua jalan keluar yang mungkin dari sebuah labirin. Metode yang dipakai adalah *Depth-First Search* secara rekursif yang dikombinasikan dengan teknik *backtracking* (runut balik). Bedanya dengan pencarian BFS yang menyebar rata, DFS ini sifatnya menelusuri satu rute sedalam-dalamnya secara berurutan sampai mentok menemukan jalan buntu atau menyentuh titik tujuan. Karena soal minta cari semua kemungkinan "jalur sederhana" yakni aturannya tidak boleh menginjak kotak yang sama lebih dari satu kali dalam satu kali perjalanan. Teknik *backtracking* itu setiap kali satu rute selesai dicek atau mentok, algoritma akan melangkah mundur, menghapus jejak langkahnya, lalu mencoba masuk ke cabang belokan lain yang belum dieksplorasi.

Untuk alur kerjanya, program menyiapkan array grid untuk memetakan peta, serta array boolean visited yang awalnya diisi false semua sebagai papan penanda jejak langkah. Pas fungsi dfs dipanggil berawal dari titik *start*, hal pertama yang dicek adalah apakah posisi kita sekarang sudah sama dengan koordinat *finish*.

Kalau iya, variabel *total_jalur* ditambah satu, lalu program langsung mengeksekusi *return*. Kalau belum sampai tujuan, kotak tempat kita berdiri sekarang ditandai dulu menjadi true di array *visited* supaya tidak terinjak dua kali secara tidak sengaja. Setelah itu, program mengecek 4 arah di sekitarnya. Jika petak *next* aman (masih di dalam batas, berupa jalan 0, dan berstatus false), maka fungsi *dfs* akan memanggil dirinya sendiri masuk ke petak baru tersebut. Nah, setelah semua cabang dari titik itu selesai dicek, status *visited* kotak tersebut diubah kembali menjadi false. Proses membuka kembali jalur ini yang dinamakan *backtracking*, sehingga jalan tersebut bisa dilewati kembali oleh rute alternatif lain. Asumsi adalah pergerakan murni hanya lurus horizontal dan vertikal tanpa ada pergerakan menyilang atau diagonal.

Untuk **analisis waktu eksekusinya**, harus lihat mekanisme percabangan rekursifnya. Di setiap petak, program selalu mengecek 4 pilihan arah secara konstan melalui perulangan *for*. Karena pencarian selalu bercabang 4 secara terus-menerus dengan rute maksimal seluas R dikali C , maka kompleksitas waktunya menembus $O(4^{(R \times C)})$. Secara teori angka eksponensial ini sangat gede, namun batasan soal mematok maksimal R dan C hanya 7 (luas maksimal 49 kotak). Ukuran peta yang kecil dan banyaknya dinding yang memotong percabangan, perhitungannya tidak akan overflow sehingga ga nyentuh *Time Limit* 2.0 s.

Selanjutnya **untuk kompleksitas ruang**, algoritma hanya mendeklarasikan array statis *grid* dan *visited* yang dikunci di ukuran 10x10. Saat pencarian rekursif berjalan, memori internal komputer akan menampung riwayat pemanggilan fungsi (*call stack*). Karena adanya *backtracking* dan aturan larangan injak kotak yang sama dua kali, tumpukan memori maksimalnya hanya akan mentok max total luas labirin. Dari ukuran array yang sangat kecil ditambah kedalaman rekursif, kompleksitas ruangnya adalah $O(R \times C)$. Alokasi ini irit, dijamin terhindar dari *crash* akibat memori penuh (*stack overflow*), dan lolos batasan *Memory Limit* 64 MB.

Terakhir, **pembuktikan logika** pada contoh kasus 5x6. Mulai dari koordinat (0,0), algoritma menelusuri satu cabang hingga ujung. Jika terjebak di jalan buntu seperti pada sel (0,5), program langsung melakukan *backtracking* (mundur ke petak sebelumnya) lalu coba cabang belokan lain. Siklus maju dan mundur ini dieksekusi terus-menerus. Setiap kali rambatan berhasil menembus koordinat target (4,5), nilai total_jalur bertambah 1, lalu rutenya dibongkar mundur untuk mencari jalan lain. Setelah semua kombinasi belokan diuji, program selesai dan mencetak total 4 rute valid, membuktikan akurasi algoritma sudah sinkron dengan modul.

SOAL 5

Tree Traversal

A. Source Code

Tabel 5. Source Code Soal 5

1	#include <iostream>
2	#include <string>
3	#include "RedBlackTree.h"
4	
5	using namespace std;
6	
7	void cetakPreorder(const RedBlackTree::Node* node)
	{
8	if (node->isNil) return;
9	cout << node->key << " ";
10	cetakPreorder(node->left);
11	cetakPreorder(node->right);
12	}
13	
14	void cetakInorder(const RedBlackTree::Node* node)
	{
15	if (node->isNil) return;
16	cetakInorder(node->left);
17	cout << node->key << " ";
18	cetakInorder(node->right);
19	}
20	
21	void cetakPostorder(const RedBlackTree::Node*
	node) {
22	if (node->isNil) return;
23	cetakPostorder(node->left);

```

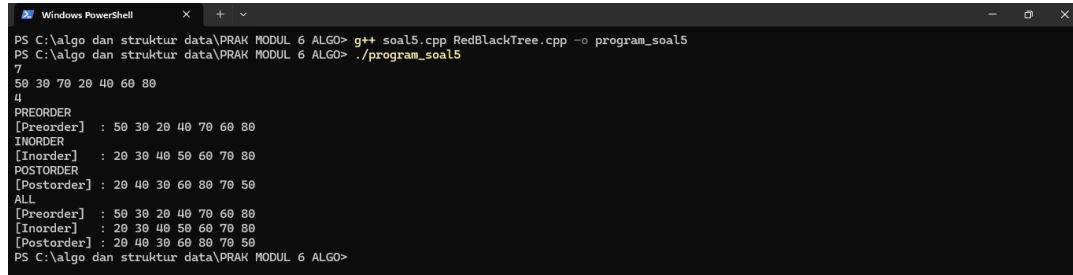
24     cetakPostorder(node->right);
25     cout << node->key << " ";
26 }
27
28 int main() {
29     RedBlackTree pohon;
30
31     int n;
32     cin >> n;
33
34     for (int i = 0; i < n; i++) {
35         int angka;
36         cin >> angka;
37         pohon.insert(angka);
38     }
39
40     int q;
41     cin >> q;
42
43     for (int i = 0; i < q; i++) {
44         string perintah;
45         cin >> perintah;
46
47         if (pohon.empty()) {
48             cout << "Tree kosong. Tidak ada yang
bisa ditampilkan.\n";
49         }
50         else {
51             if (perintah == "PREORDER") {
52                 cout << "[Preorder]   : ";

```

53	cetakPreorder(pohon.root());
54	cout << "\n";
55	}
56	else if (perintah == "INORDER") {
57	cout << "[Inorder] : ";
58	cetakInorder(pohon.root());
59	cout << "\n";
60	}
61	else if (perintah == "POSTORDER") {
62	cout << "[Postorder] : ";
63	cetakPostorder(pohon.root());
64	cout << "\n";
65	}
66	else if (perintah == "ALL") {
67	cout << "[Preorder] : ";
68	cetakPreorder(pohon.root());
69	cout << "\n";
70	
71	cout << "[Inorder] : ";
72	cetakInorder(pohon.root());
73	cout << "\n";
74	
75	cout << "[Postorder] : ";
76	cetakPostorder(pohon.root());
77	cout << "\n";
78	}
79	}
80	}
81	
82	return 0;

83	}
----	---

B. Output Program



```
PS C:\algo dan struktur data\PRAK MODUL 6 ALGO> g++ soal5.cpp RedBlackTree.cpp -o program_soal5
PS C:\algo dan struktur data\PRAK MODUL 6 ALGO> ./program_soal5
7
50 30 70 20 40 60 80
4
PREORDER
[Preorder] : 50 30 20 40 70 60 80
INORDER
[Inorder] : 20 30 40 50 60 70 80
POSTORDER
[Postorder] : 20 40 30 60 80 70 50
ALL
[Preorder] : 50 30 20 40 70 60 80
[Inorder] : 20 30 40 50 60 70 80
[Postorder] : 20 40 30 60 80 70 50
PS C:\algo dan struktur data\PRAK MODUL 6 ALGO>
```

Gambar 5. Screenshot Output Program Soal 5

C. Pembahasan

Algoritma ini bertujuan untuk membangun struktur data pohon biner seimbang, yakni *Red-Black Tree* (RBT), dan melakukan penelusuran (*traversal*) untuk mencetak isinya. Mekanisme penelusurannya menggunakan metode rekursif *Depth-First Search* (DFS) yang dibagi menjadi tiga teknik: *Preorder* (cetak *root*, telusuri kiri, telusuri kanan), *Inorder* (telusuri kiri, cetak *root*, telusuri kanan), dan *Postorder* (telusuri kiri, telusuri kanan, cetak *root*).

Untuk cara kerjanya, program pertama-tama membaca *N* angka lalu memasukkannya satu per satu ke dalam objek pohon menggunakan fungsi *insert*. Angka yang duplikat otomatis diabaikan agar tidak ada *node* ganda. Setelah struktur pohon terbentuk, program mengeksekusi perulangan sebanyak *Q* kali untuk membaca perintah *query*. Jika terdeteksi bahwa pohon kosong, program akan mencetak pesan penolakan. Namun jika ada isinya, algoritma menggunakan blok pengecekan *if-else* untuk menyesuaikan *string* perintah dengan fungsi cetak rekursif yang dipanggil. Asumsi pada kode ini adalah proses *balancing* (seperti rotasi dan pewarnaan *node*) dieksekusi secara tersembunyi di dalam *header* *RedBlackTree.h*, dan penanda ujung daun (*null/leaf*) dikendalikan menggunakan properti boolean *isNil*.

Untuk menganalisis waktu eksekusinya, proses terbagi menjadi dua fase. Fase pertama adalah memasukkan N data ke dalam pohon dan memakan waktu $O(N \log N)$, karena setiap penambahan *node* baru memerlukan pengecekan dan penyesuaian tinggi pohon agar tetap seimbang. Fase kedua adalah pencetakan traversal. Mengunjungi seluruh *node* di pohon membutuhkan waktu $O(N)$. Karena ada Q buah *query*, waktu proses bagian ini adalah $O(Q \times N)$. Jika digabung, total kompleksitas waktunya adalah $O(N \log N + Q \times N)$. Berdasarkan batasan soal, N maksimal 1000 dan Q maksimal 20. Dalam skenario terburuk, program hanya menjalankan sekitar puluhan ribu instruksi komputasi. Jumlah ini sangatlah ringan sehingga prosesnya dijamin super cepat dan aman jauh di bawah *Time Limit* 1.0 s.

Selanjutnya untuk kompleksitas ruang, memori yang terpakai dialokasikan untuk menyimpan N buah *node* pada struktur *Red-Black Tree* dan memori tumpukan fungsi rekursif (*call stack*). Mengingat sifat RBT yang selalu melakukan *auto-balancing*, tinggi maksimal pohon dipastikan akan selalu mendekati $\log N$. Artinya, kedalaman tumpukan memori rekursifnya sangat dangkal. Berdasarkan ruang untuk *node* pohon dan tumpukan rekursif tersebut, kompleksitas ruang totalnya adalah $O(N)$. Untuk menampung maksimal 1000 elemen, alokasi memori ini sangatlah kecil, jadi terhindar dari *stack overflow*, dan sudah pasti 100% lolos dari batas *Memory Limit* 64 MB.

Terakhir, **pemuktikan logikanya** melalui contoh kasus berisi 7 input angka (50, 30, 70, 20, 40, 60, 80). Angka pertama (50) akan ditetapkan sebagai *root*. Angka yang lebih kecil akan otomatis dilempar ke cabang subtree kiri dan yang lebih besar ke kanan. Saat program diminta mengeksekusi perintah INORDER, algoritma akan memaksa penelusuran sampai ke ujung kiri terbawah terlebih dahulu, baru mencetak *root*, lalu berpindah ke kanan. Karena mekanisme ini, *output* dari fungsi Inorder pada *Red-Black Tree* akan selalu tercetakurut dari kecil ke besar secara otomatis (yakni 20 30 40 50 60 70 80). Bukti cetakan yang berurutan ini memastikan bahwa logika kerangka RBT dan fungsi traversal rekursif sudah sinkron dengan standar modul.

TAUTAN GITHUB

[DSA26-ULM/module-6-graph-and-tree-userdyaaa: dsa26-ulm-classroom-3c2beb-module-6-graph-and-tree-module-6-graph-and-tree created by GitHub Classroom](#)